

1. What is Synchronization, and why is it needed? ★ ★ ★

Computer Example

Suppose two threads update the same bank account.

```
Balance = 1000

Thread A
Withdraw 100

Thread B
Deposit 50
```

If both run simultaneously:

```
A reads 1000
B reads 1000

A writes 900
B writes 1050
```

Final Balance = 1050 ✕

Correct should be:

```
1000
-100
+50
=950
```

Synchronization ensures only one thread updates the balance at a time.

Definition

Synchronization is the process of **coordinating multiple threads or processes** so they can safely access shared resources without causing incorrect results.

Why is it Needed?

Because multiple threads may share:

- Variables
- Files
- Database records
- Memory
- Printer
- Network socket

Without synchronization:

- Data corruption
- Wrong calculations
- Crashes
- Inconsistent state

Interview Answer (30 seconds)

Synchronization is the coordination of multiple threads or processes to ensure safe access to shared resources. It prevents problems like race conditions and data corruption by ensuring operations occur in the correct order or by allowing only one thread into a critical section at a time.

Follow-up

Q: Is synchronization only for multithreading?

Answer:

No.

It is used for:

- Multiple threads
- Multiple processes
- Distributed systems
- Databases

Anywhere shared resources exist.

2. What is a Critical Section? ★ ★ ★

Real-Life Example

Imagine a washroom with only **one key**.

Everyone can stand outside.

But only the person holding the key may enter.

The washroom is the **critical section**.

The key acts like a **mutex**.

Structure

Entry **Section**

↓

Critical **Section**

↓

Exit **Section**

↓

Remaining Code

Example

```
lock();  
  
counter++;  
  
unlock();
```

Only

```
counter++;
```

is the critical section.

Definition

A critical section is the portion of a program where shared resources are accessed or modified.

Only one thread should execute it at a time.

Interview Answer

A critical section is the part of a program that accesses shared data or resources. Since concurrent access can lead to incorrect results, only one thread or process should execute the critical section at any given time.

Follow-up

Q: Is every function a critical section?

No.

Only code that accesses shared resources.

3. What are the Three Requirements of the Critical Section Problem? ★★ ★

The OS must satisfy **three conditions**.

Requirement 1: Mutual Exclusion

Meaning

Only one **thread** can enter the **critical** section.

Interview Answer

Mutual exclusion means that only one process or thread can execute the critical section at a time, preventing simultaneous access to shared resources.

Requirement 2: Progress

Meaning

If nobody is inside the critical section,
don't block everyone unnecessarily.
Someone waiting should be allowed to enter.

Interview Answer

Progress means that if no thread is executing inside the critical section, one of the waiting threads should eventually be selected to enter. The system should not delay access unnecessarily.

Requirement 3: Bounded Waiting

Meaning

Nobody should wait forever.

Interview Answer

Bounded waiting guarantees that after a thread requests entry into the critical section, there is a limit on how many other threads can enter before it gets its turn. This prevents starvation.

Easy Memory Trick

MEP

M → Mutual Exclusion

E → Everyone eventually gets a chance (Progress)

P → Prevent infinite **waiting** (**Bounded Waiting**)

Interview Follow-up

Q: Which requirement prevents starvation?

Answer:

Bounded Waiting.

Q: Which requirement prevents two threads from entering together?

Answer:

Mutual Exclusion.

Q: Which requirement prevents unnecessary blocking?

Answer:

Progress.

4. What is a Race Condition? ★ ★ ★

One of the most asked interview questions.

Real-Life Example

Imagine two people editing the same Google Doc.

Original:

```
Salary = 1000
```

Person A changes it to

```
900
```

Person B changes it to

```
1050
```

Whoever saves last wins.

One update is lost.

That's a race condition.

```
6
```

Why does it happen?

Because

```
Read
```

```
↓
```

```
Modify
```

```
↓
```

```
Write
```

is not atomic.

Another thread interrupts.

Definition

A race condition occurs when multiple threads access and modify shared data simultaneously, and the program's result depends on the timing or order of execution.

How to Prevent It?

Use synchronization:

- Mutex
- Semaphore
- Monitor
- Spinlock
- Atomic operations

Interview Answer

```
A race condition occurs when multiple threads access shared data concurrently and at least one modifies it without proper synchronization. As a result, the program's output depends on the execution order, leading to unpredictable or incorrect results.
```

Interview Follow-up

Q: Does every race condition produce wrong output?

No.

Sometimes the output happens to be correct by chance.

The problem is that it's **unpredictable**.

Q: Is race condition possible with one thread?

No.

A race condition requires **multiple concurrent execution paths** (multiple threads or processes) accessing shared data.

Interview Summary (Remember These)

1. Synchronization

Synchronization coordinates multiple threads or processes to safely access shared resources and prevents data inconsistency.

2. Critical Section

A critical section is the part of a program where shared data is accessed or modified. Only one thread should execute it at a time.

3. Three Requirements

- **Mutual Exclusion:** Only one thread in the critical section.
- **Progress:** If the critical section is free, a waiting thread should be allowed to enter without unnecessary delay.
- **Bounded Waiting:** Every waiting thread eventually gets a chance; no starvation.

4. Race Condition

A race condition occurs when concurrent access to shared data causes the program's result to depend on execution timing, leading to unpredictable or incorrect behavior.

These four concepts are the foundation for understanding **mutexes, semaphores, spinlocks, monitors, and classic synchronization problems**, which build on the same idea of safely coordinating concurrent access to shared resources.

***** take 5 min break *****